

GeoGraphRAG: A graph-based retrieval-augmented generation approach for empowering large language models in automated geospatial modeling

Jianyuan Liang, Shuyang Hou, Haoyue Jiao, Yaxian Qing, Anqi Zhao, Zhangxiao Shen, Longgang Xiang, Huayi Wu^{*}

State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing, Wuhan University, Wuhan, China

ARTICLE INFO

Keywords:

Large Language Model (LLM)
Geospatial modeling
Domain knowledge injection
Graph-based retrieval-augmented generation
Intelligent generation

ABSTRACT

Geospatial modeling aims to integrate multiple geospatial data sources and geoprocessing functions based on user application demands to address complex geospatial challenges. While large language models (LLMs) have shown remarkable capabilities in semantic understanding and task planning, their limited geospatial domain knowledge still hinders their efficiency in geospatial modeling. Therefore, there is an urgent need to integrate geospatial knowledge with the inherent semantic understanding capabilities of LLMs, enabling their effective application in specialized geospatial domains. In this study, we introduce GeoGraphRAG, a graph-based retrieval-augmented generation (RAG) approach tailored for geospatial modeling. Specifically, GeoGraphRAG utilizes LLMs as intelligent agents to identify user demands, retrieve relevant subgraphs from external graph knowledge base, and injects both structural and semantic information into the LLM. Through the graph-driven solution planning phase, GeoGraphRAG automatically generates modeling solutions and geospatial code in response to specific user demands. The proposed method is evaluated against different baseline approaches across various LLMs based on the benchmark dataset designed for geospatial modeling, and a representative application scenario is presented to illustrate its practical effectiveness. Our experimental results demonstrate that the injection of graph-structured knowledge improves both the efficiency and interpretability of LLM-based geospatial modeling, thereby confirming the effectiveness of our proposed approach. GeoGraphRAG represents the first attempt to introduce graph-based RAG into geospatial modeling and provides novel insights for future research and practical applications in this field.

1. Introduction

In the GIS domain, geospatial modeling is an essential approach to understand geographical phenomena, tackle complex geospatial challenges, and support decision-making (Chen et al., 2020; He et al., 2023; Li et al., 2021). In recent years, cloud platforms such as Google Earth Engine (GEE), PieEngine, and OGE have been established to support geospatial modeling by integrating vast data resources and geoprocessing methods into analysis-ready services that are delivered through cloud-native APIs (Gao et al., 2022; Wu et al., 2024). Researchers can develop and execute geoprocessing scripts in the cloud using their expert knowledge, without concerns about common issues like runtime environments, data acquisition, or service quality (Liang et al., 2024). Consequently, constructing geoprocessing scripts for geospatial modeling has gradually gained widespread adoption (Li and Ning, 2023; Y. Zhang et al., 2024b). Nevertheless, given the learning

curve, complexity, and technical barriers of developing geospatial code, the demand for automated generation of geoprocessing scripts based on user demands is steadily increasing (Hou et al., 2025).

Recent progress in Generative Artificial Intelligence (GAI) and Large Language Models (LLMs) has brought new opportunities for automated geospatial modeling by reducing reliance on manual expertise, accelerating model development, and enhancing the reproducibility of spatial analyses (Cao et al., 2023; M. Chen et al., 2024a). LLMs have demonstrated strong capabilities in code generation and exhibited notable task understanding and planning abilities for geospatial modeling (Gramacki et al., 2024; Hou et al., 2024b; Mooney et al., 2023). For instance, LLMs can generate simple geospatial analysis code using libraries such as ArcPy, GeoPandas, and the GEE Client Library. During interactions with the LLM, users provide their modeling demands—such as temporal and spatial scopes, analytical objectives, and data sources—in natural language as prompts. The LLM interprets these inputs and generates

^{*} Corresponding author.

E-mail address: wuhuay1@whu.edu.cn (H. Wu).

<https://doi.org/10.1016/j.jag.2025.104712>

Received 3 April 2025; Received in revised form 16 June 2025; Accepted 2 July 2025

Available online 10 July 2025

1569-8432/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

corresponding modeling solutions, which typically define a workflow composed of a series of processing steps (Y. Zhang et al., 2024b). For instance, given a user prompt such as “Calculate the NDVI image of Wuhan in 2024 using Landsat imagery”, along with supplementary instructions (e.g., the programming language and modeling environment) and prompting strategies (e.g., Chain-of-Thought (CoT)) (Wei et al., 2022), LLMs can generate a step-by-step modeling solution, including processing steps like “image loading”, “image cropping”, “NDVI calculation”, and “result export”. This modeling solution can then serve as a blueprint for generating the corresponding implementation code (Wei et al., 2024). However, current LLMs are primarily pre-trained on general corpora, with limited knowledge in the geospatial domain (X. Gu et al., 2024b; Lin et al., 2023). When confronted with more complex geospatial modeling requirements, they may encounter issues such as “code refusal” or “hallucination” (Liu et al., 2024). Specifically, LLMs may fail to generate a reliable and coherent modeling solution, or produce either a modeling solution or geospatial code that appear correct but is actually erroneous.

To address this challenge, Retrieval-Augmented Generation (RAG) has emerged as a promising approach that injects external knowledge from domain-specific knowledge bases, and bridges the knowledge gaps of LLMs in vertical domains, without altering the fundamental architecture of the model (Lewis et al., 2020). The core of RAG lies in designing a retrieval component that can dynamically queries large text corpora and incorporates relevant factual knowledge into prompt engineering, thereby enhancing both the relevance and accuracy of the reasoning and generation processes in LLMs (Sun et al., 2024; Xia et al., 2024). In the context of RAG, while text-based retrieval is effective, graph structures offer a more efficient alternative for organizing knowledge (Chein and Mugnier, 2008). Such graph-structured data (e.g., knowledge graphs) enables the explicit representation of relationships between entities, reduces information redundancy, and allows for more flexible updates (Peng et al., 2024; Robinson et al., 2015). Recent studies have explored their integration into RAG frameworks as external knowledge sources, a direction commonly referred to as graph-based RAG.

However, research on applying graph-based RAG on geospatial modeling is still in its early stages, and three key challenges remain to be addressed. Firstly, a graph structure containing dense domain knowledge is required to guide geospatial modeling. Ideally, the retrieval objective is to extract a subgraph that contains diverse processing steps and their relationships, which collectively represents a modeling solution that fulfills the user demand. Secondly, even with a well-structured graph serving as the external knowledge base, an efficient retriever remains essential in graph-based RAG to support multi-hop reasoning over the graph. Specifically, the graph may contain multiple paths that connect the given starting node to the ending node. In the context of geospatial modeling, these paths correspond to alternative sequences of processing steps, and the appropriate selection of such steps is crucial for generating an optimal modeling solution. Thirdly, a strategy should be developed to convert the retrieved graph information into a specific format that preserves the relational dependencies, thereby ensuring that the structured data can be effectively interpreted and utilized by the language model. Once formatted, the graph data is combined with the query and provided to the LLM as input for reasoning and generation.

In this study, we propose GeoGraphRAG, a graph-based RAG approach designed for automated geospatial modeling. Specifically, this study adopts S-GMKG (Liang et al., 2025), a domain knowledge graph that consolidates a wide range of expert-defined processing steps and their relationships, as the external graph-based knowledge source (see Section 2 for details). Building upon the S-GMKG, GeoGraphRAG employs the LLM as an autonomous agent to interact with the knowledge graph. The LLM first extracts key concepts from the user query expressed in natural language, matches them to candidate entities for graph retrieval, and identifies the subgraph that represents a modeling solution aligned with the user demand. Subsequently, the subgraph is

iteratively structured into prompt contexts through a graph-to-text conversion process, enabling the LLM to generate a complete modeling solution along with the corresponding implementation code. We evaluate GeoGraphRAG against several baseline methods with a series of state-of-the-art (SOTA) LLMs on the benchmark dataset, assessing both the generated modeling solutions and the corresponding implementation code. The results demonstrate that GeoGraphRAG effectively leverages the inherent capabilities of LLMs to retrieve graph structures and utilize graph information to support automated geospatial modeling, without requiring fine-tuning of the underlying black-box models. Furthermore, GeoGraphRAG can improve model interpretability through knowledge-based responses and explicit reasoning explanations.

The rest of this paper is organized as follows. Section 2 introduces background on the S-GMKG and the graph-based RAG. Section 3 details the design and implementation of the proposed GeoGraphRAG. Section 4 presents the experimental evaluation and an illustrative application case. The discussion and conclusion of this study are provided in Section 5 and Section 6.

2. Preliminaries

2.1. Brief introduction to S-GMKG

S-GMKG is a knowledge graph that integrates modeling processes extracted from heterogeneous geoprocessing scripts (Liang et al., 2025). Following the “LLM4KG” application paradigm (Pan et al., 2024), it is automatically constructed by leveraging LLMs to parse geoprocessing scripts, extract the implicit the input–output data and functional modules embedded in the code, and convert them as semantic units within a graph structure.

As shown in Fig. 1, two types of semantic entities—functional step and IO-data—are defined in S-GMKG, which represent functional modules and input/output data, respectively. A functional module is equivalent to a processing step implemented by a code snippet comprising one or more APIs. Each functional step entity contains a descriptive step name indicating its functionality, and an optional *CodeSnippet* attribute that provides reference code for its implementation. The functional step and IO-data entities are connected through *transfer* and *invoke* relations. Specifically, the *transfer* relation captures the flow of data, where IO-data entities serve as inputs to a functional step entity or are generated as its outputs. The *invoke* relation, on the other hand, defines the execution order between functional steps. As different functional step and IO-data entities are frequently reused across different modeling processes, their interconnections collectively form S-GMKG as a knowledge-intensive graph structure.

Since each connection in S-GMKG is directly extracted from geoprocessing scripts—developed by domain experts to specify processing steps and their logical order—S-GMKG inherently facilitates the integration of expert knowledge. Theoretically, each geoprocessing script, as an implementation of a geo-analysis model, naturally corresponds to a subgraph within S-GMKG. Conversely, any subgraph extracted from S-GMKG may serve as a candidate modeling solution for an existing or potential geo-analysis task. These structural characteristics make S-GMKG a promising graph-based knowledge source for RAG in geospatial modeling, as they enable the representation of modeling solutions in the form of subgraphs tailored to specific analytical goals.

A specific example is illustrated in Fig. 1. The user demand, “Calculate the NDVI image of Wuhan in 2024 using Landsat imagery”, can be mapped to a path from one IO-data entity (Landsat Image) to another (NDVI Image) in S-GMKG. The nodes and edges along this path are extracted as triples, which depict the involved data source, processing steps, supplementary functionality description and reference code snippets, as well as their combinational logic. These triples can be further verbalized into natural language statements to support LLM-based reasoning. Therefore, the key to apply S-GMKG for geospatial

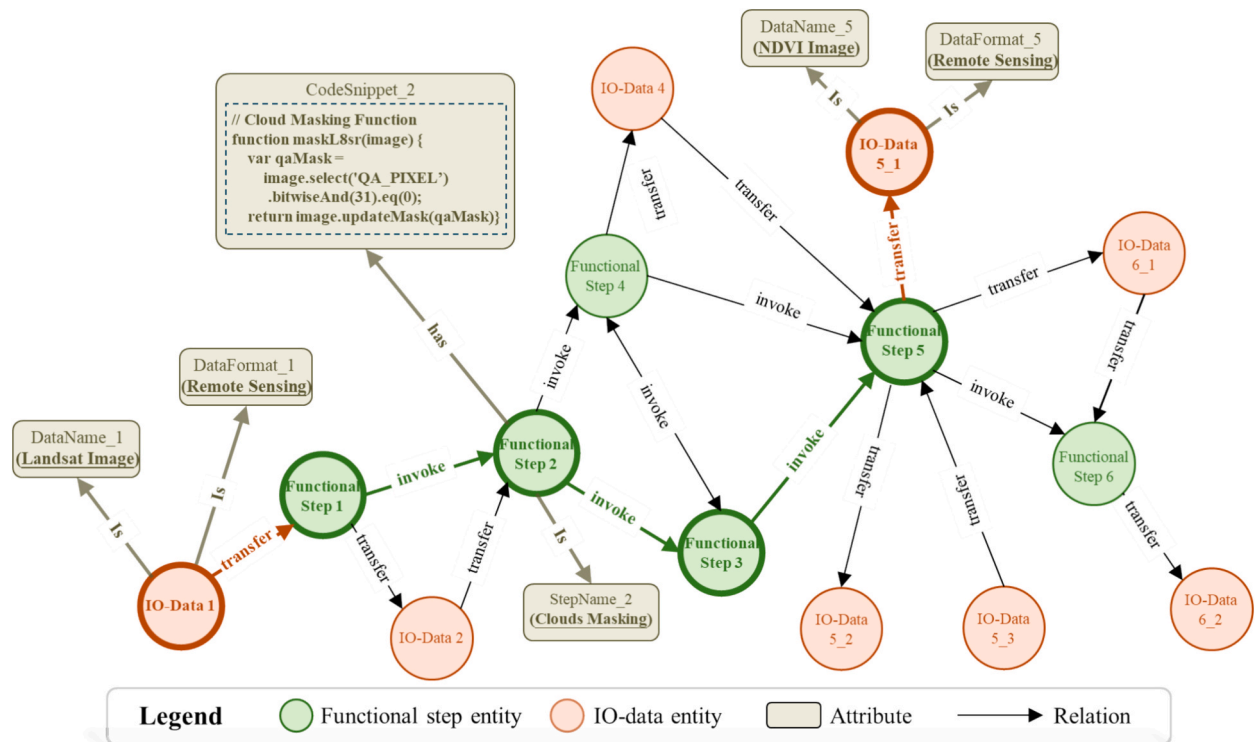


Fig. 1. Structure of the S-GMKG. Green nodes represent the functional steps, orange nodes represent the IO-data, and light brown rectangles represent attributes. Arrows indicate *transfer*/*invoke* relations among functional step and IO-data entities. Bolded items highlight selected objects for the example (Note that only a subset of the relation and attributes is presented for clarity). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

modeling lies in developing an effective method for retrieving the candidate subgraph that satisfies user demands.

2.2. Graph-based RAG with LLM

In contrast to traditional RAG, which retrieves unstructured textual content from large-scale corpora, graph-based RAG captures structured relational knowledge by retrieving graph elements—including nodes, edges, paths, or subgraphs—from an external graph-based knowledge source (Peng et al., 2024). The ability to accurately retrieve relevant entities and relationships is crucial to ensure the quality and contextual relevance of LLM-generated outputs (Luo et al., 2024). Once high-quality graph information is retrieved, graph-based RAG can deliver more accurate, consistent, and context-aware responses (Y. Zhang et al., 2024a).

Considerable efforts have been dedicated to extracting relevant and high-quality graph information from graph structure to support LLM reasoning with retrieved information. For instance, Delile et al. (2024) applied the shortest path algorithm to connect entities parsed from the user query and used the subgraph as the graph retrieval result. Kim et al. (2023) directly utilized LLMs to generate potential entities and autonomously selected their top-k relevant relations from the knowledge graph, thereby retrieving triples for LLM reasoning. Luo et al. (2024) extracted subgraphs from the knowledge graph by using candidate relation pairs generated through prompt engineering. To optimize the relevance of the selected relation pairs to the query, this work further proposed a fine-tuning strategy that uses relation pairs as training signals to enable the language model to learn the semantic relationships from the knowledge graph. Besides, domain-specific adaptations are essential to enhance the efficiency and applicability of graph-based RAG. For instance, D. Li et al. (2024) proposed a self-aware knowledge retrieval strategy that can select subgraph information closely related to given diseases from the medical knowledge graph for clinical

decision support. Jin et al. (2024) defined four general interaction rules that allow LLMs to reason directly over domain-specific knowledge graphs, thereby enhancing knowledge-driven question-answering across various domains.

In the field of GIS, RAG is primarily employed for more accurate geospatial knowledge question-answering that covers multiple scenarios such as geospatial commonsense, disaster warning, and urban sensing (Deng et al., 2024; Manvi et al., 2023; Ning et al., 2025). Although research on applying graph-based RAG to geospatial modeling is still limited, several studies have showcased the potential of graph-structured information in enhancing LLMs' capabilities for such tasks. For instance, Wei et al. (2024) employed invocation chains of GIS tools as prompts to guide a fine-tuned LLM in constructing solution plans and achieved results that surpass baseline performance in both general and complex tasks. Z. Li & Ning, (2023) leveraged LLMs to generate solution graphs, which were subsequently used as guides to drive automatic geospatial code generation. Nevertheless, these studies primarily rely on the proficiency of LLMs in geospatial modeling and their task planning capabilities, which poses risks to the reliability and interpretability of their responses.

3. Methodology

3.1. Overall framework

The objective of GeoGraphRAG is to enhance the geospatial modeling capabilities of LLMs through a graph-based RAG framework built upon S-GMKG. As shown in Fig. 2, GeoGraphRAG consists of three core modules: the Demand Identifier, the Graph Retriever, and the Solution Planner, with the S-GMKG serving as its foundation. Given user demands described in natural language, GeoGraphRAG retrieves relevant graph information from the S-GMKG and collaborates with LLMs to generate modeling solutions and corresponding implementation code.

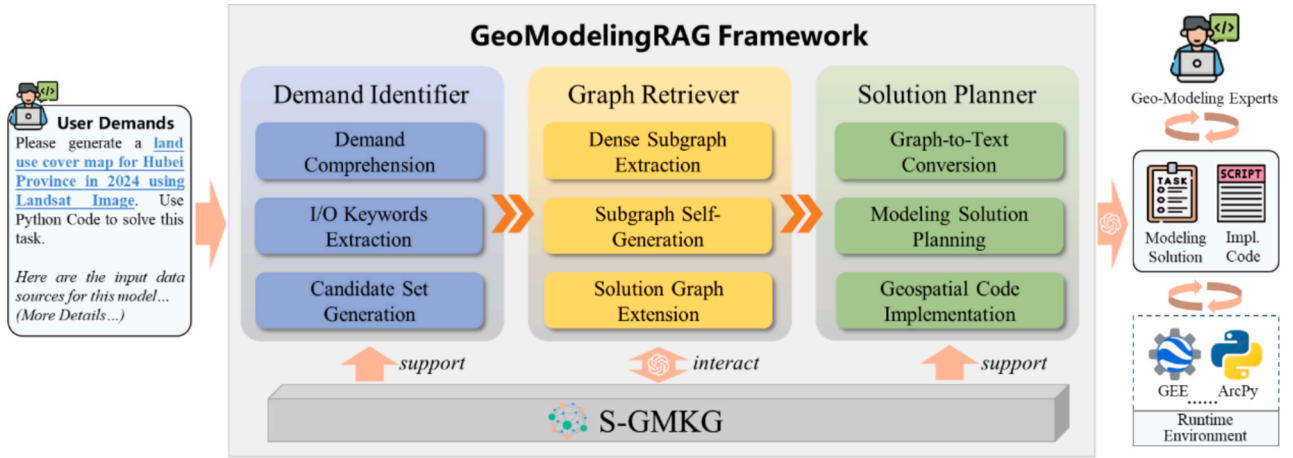


Fig. 2. Framework of the GeoGraphRAG for automated geospatial modeling.

The Demand Identifier (Section 3.2) utilizes the LLM as an intelligent agent to parse the user demand and extract key descriptive terms that represent potential inputs and target outputs to form a candidate set. In the Graph Retriever (Section 3.3), these candidates are semantically mapped to IO-data entities in the S-GMKG. The matched entities serve as the source and target nodes for path retrieval, and a corresponding subgraph (referred to as the solution graph) that aligns with the user demand for geospatial modeling is extracted as the response. In the Solution Planner (Section 3.4), this solution graph is converted into structured prompt texts through a rule-based template, which are then used to guide the LLM in generating a tailored modeling solution and the corresponding implementation code.

3.2. Demand identification

This module aims to identify potential input and output keywords from a user demand expressed in natural language, in order to support subsequent graph retrieval from the S-GMKG. Typically, descriptive terms can be directly extracted from the user demand or logically inferred from its context, while the clarity and specificity of the user demand determine the efficiency of extracting the necessary information. For instance, in the user demand “Obtain the land use/cover map of

Hubei Province for the year 2024”, the target output, “land use/cover map”, is explicitly identifiable. While the data source for this task is not specified, expert knowledge suggests that remote sensing imagery (e.g., Landsat, Sentinel) could serve as potential inputs. Thus, a candidate set including terms like “Landsat imagery” and “land use/cover map” is created for querying the S-GMKG. Notably, although the temporal scope (i.e., year 2024) and spatial context (i.e., Hubei Province) are relevant for final code generation, they are excluded in this module, as their contribution to subgraph extraction is minimal.

The LLM is driven to interpret three categories of user demands: explicit, semi-explicit, and implicit, each varying in levels of abstraction and complexity. As illustrated in Fig. 3, the LLM will autonomously determine the type of the user demand through prompt-based guidance. If the user demand considered explicit, the LLM will directly parse the description and extract keywords representing potential inputs and the target output, thus forming the candidate set. For semi-explicit demand, where the input or output is implied but not explicitly stated, the LLM supplements the original user demand using the CoT approach. By reasoning over factors such as the modeling objectives, process logic, and data types, the model infers the intended geospatial modeling goal and generates the missing input or output terms accordingly.

However, when the user demand is considered vague, which

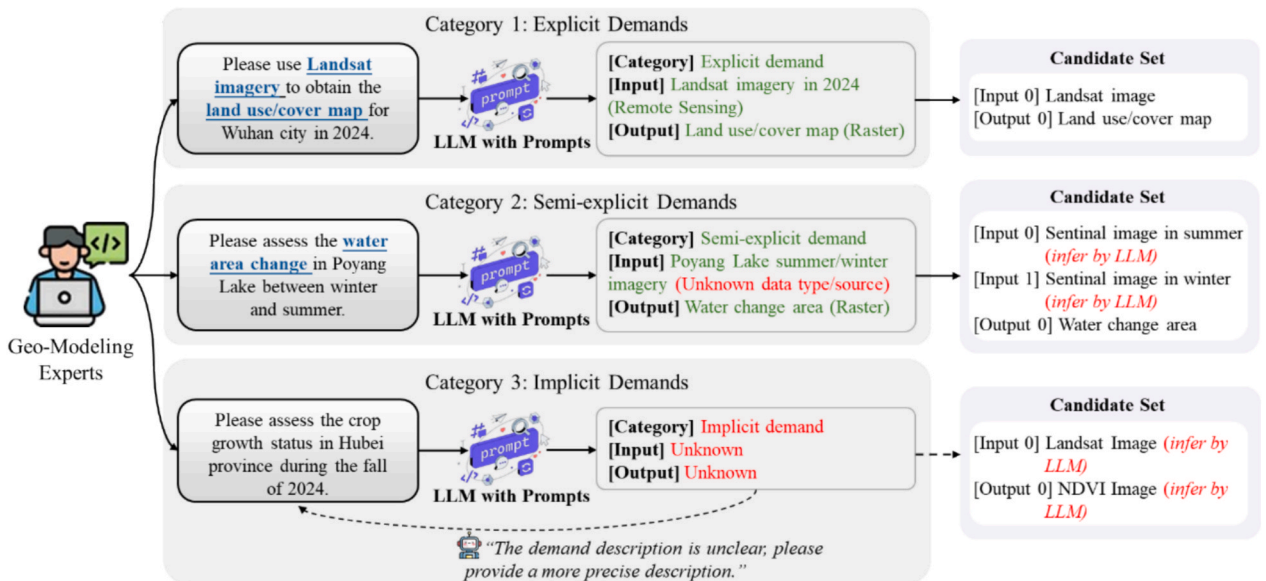


Fig. 3. Examples of handling three categories of user demand in geospatial modeling.

indicates an implicit demand, an interactive mode will be activated. In this mode, the user is prompted to either refine the original query or allow the LLM to creatively infer and generate potential input/output keywords based on its knowledge on geospatial modeling, even though this may carry the risk of deviating from the original intent of the user. The detailed prompt template used to guide the LLM in demand identification is provided in the Appendix, with the example in Fig. 3 incorporated for few-shot prompting.

3.3. Subgraph retrieval

To compensate for the domain knowledge gaps in LLMs and support their autonomous generation of modeling solutions and implementation code for geospatial modeling, a core challenge lies in retrieving an appropriate subgraph as the solution graph based on the identified potential inputs and target output. To address this, a graph retriever is developed that consists of three sequential phases: dense graph extraction, subgraph self-generation, and solution graph extension, as illustrated in Fig. 4.

3.3.1. Dense graph extraction

The dense graph is a subset of the S-GMKG that preserves as much relevant graph information as possible in response to the user demand. To construct it, the semantic similarity between each keyword in the candidate set (introduced in Section 3.1) and all IO-data entities in the S-GMKG is firstly calculated to identify source and target entities. Sentence embeddings are generated using the BAAI/bge-m3 model (J. Chen et al., 2024b), and cosine similarity is then applied to rank the relevance scores. The top-k entity pairs with the highest semantic similarity scores are selected. For each source–target pair, the breadth-first search (BFS) algorithm is then employed to find inference paths. Since multiple valid paths may exist between a given source and target, and the shortest path is not always optimal for the geospatial modeling task, the top k paths are retained to offer diverse alternatives. As shown at the top of Fig. 4 (b), the resulting dense graph—constructed from all retrieved paths—serves to narrow the search space for subsequent stages. The pseudo-code in Fig. 5 presents the implementation workflow of the dense graph extraction algorithm, including sentence embedding, similarity computation, and path selection.

3.3.2. Subgraph self-generation

The dense graph contains multiple path combinations that connect source entities to the target entity, with the solution graph being one such combinations. As shown in the lower part of Fig. 4(b), this study adopts the in-context learning (ICL) approach (Brown et al., 2020) to iteratively select functional step entities from the dense graph to construct the solution graph. In each entity selection round, the prompt incorporates current graph context—including the previously selected subgraph—and candidate functional step entities with their textual attributes (e.g., names and descriptions), enabling the LLM to autonomously infer and choose the next functional step entity. Once selected, the entity is appended to the evolving subgraph, and pruning is simultaneously applied to exclude unreachable nodes and edges from the remaining dense graph. The iterative process continues until a complete solution graph that encompasses all valid paths from the source entities to the target entity is formed.

3.3.3. Solution graph extension

The initial solution graph constructed in the previous phase is presented as a set of triples, which encapsulate the core components of the modeling process: IO-data entities as source and target nodes, functional step entities as processing steps, and the logical relationships connecting them. Inspired by Z. Li & Ning, (2023) and Wu et al., (2024), the solution graph can be further enriched by incorporating additional IO-data entities associated with the selected functional step entities. Specifically, intermediate IO-data entities that link two functional step entities via *transfer* relation are introduced to enhance the continuity and semantic completeness of the graph, as shown in the Fig. 4(c). In addition, IO-data entities representing parameters (e.g., numerical values or string constants) can also be integrated when they are used as inputs to the functional step entities.

3.4. Graph-driven solution planning

The retrieved solution graph serves as the foundation for driving the LLM to generate a textual modeling solution along with the corresponding implementation code customized for the user demand. To enable the LLM to interpret both the semantic content and structural dependencies, the solution graph is converted into a series of natural language descriptions. As illustrated in Fig. 6, a predefined prompt template is used to streamline this conversion process. The paths

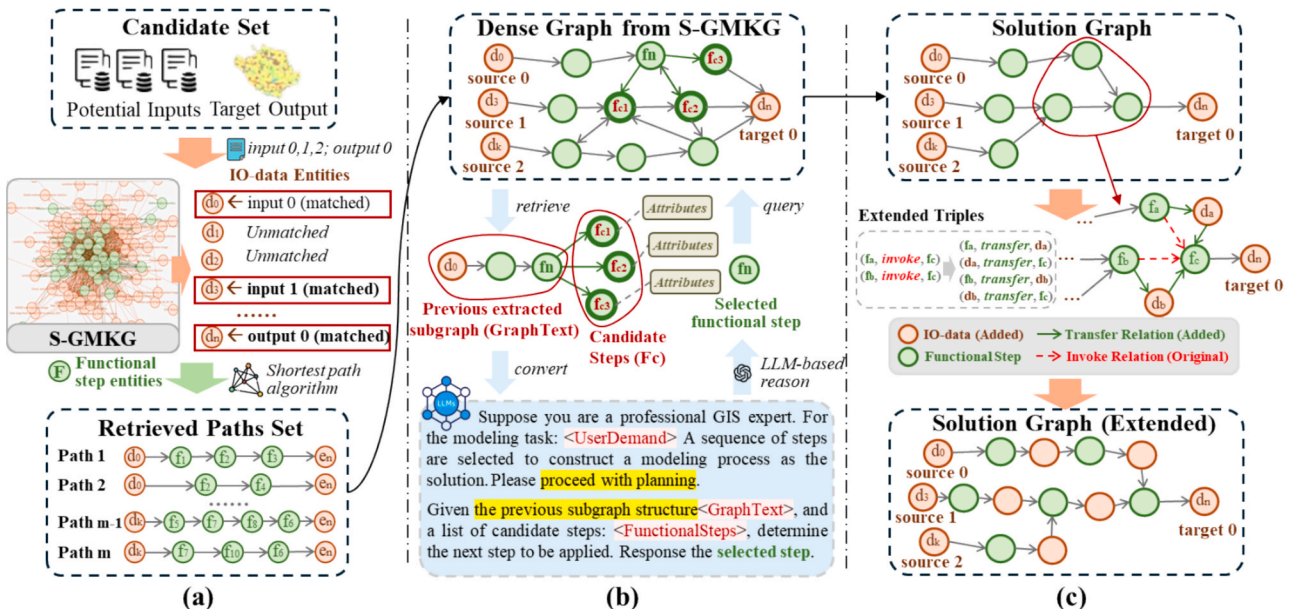


Fig. 4. Workflow of the subgraph retrieval process. (a) Dense graph extraction; (b) LLM-based subgraph self-generation; (c) Solution graph extension.

Algorithm 1. Dense Graph Extraction

Input: $Q = \{q_1, \dots, q_m\}$: keywords in the candidate set, $D = \{d_1, \dots, d_n\}$: IO-data entities,

k_1 : top entity pairs, k_2 : top paths per pair, $G(V, E)$: S-GMKG graph,

Output: DenseGraph(V' , E'): Extracted dense graph

Definitions:

$e(x)$: Sentence embedding of x obtained via embedding models (e.g., bge-m3)

$\text{sim}(q, d)$: Semantic similarity between q and d , calculated as:

$$\text{sim}(q, d) = (e(q) \cdot e(d)) / (\|e(q)\| \cdot \|e(d)\|)$$

Procedure:

- 1: **for** each $q_i \in Q$ **do**:
- 2: $e(q_i) \leftarrow \text{BGE_M3_Embed}(q_i)$
- 3: **end for**
- 4: **for** each $d_j \in D$ **do**:
- 5: $e(d_j) \leftarrow \text{BGE_M3_Embed}(d_j)$
- 6: **end for**
- 7: **for** each pair (q_i, d_j) **do**:
- 8: $\text{sim}(q_i, d_j) \leftarrow (e(q_i) \cdot e(d_j)) / (\|e(q_i)\| \cdot \|e(d_j)\|)$
- 9: **end for**
- 10: **Select** top- k_1 keyword-entity pairs (q_i, d_j) with highest $\text{sim}(q_i, d_j)$
- 11: **Let** $S_{\text{src}} \leftarrow \{d_j \mid (q_i, d_j) \text{ selected} \wedge \text{DataRole}(d_j) = \text{Input}\}$,
- 12: $S_{\text{tgt}} \leftarrow \{d_j \mid (q_i, d_j) \text{ selected} \wedge \text{DataRole}(d_j) = \text{Output}\}$
- 13: **Initialize** DenseGraph $V' \leftarrow \emptyset, E' \leftarrow \emptyset$
- 14: **for** each source_entity $s \in S_{\text{src}}$ **do**:
- 15: **for** each target_entity $t \in S_{\text{tgt}}$ **do**:
- 16: $\text{Paths} \leftarrow \text{BFS_FindAllPaths}(G, s, t)$
- 17: **Select** top- k_2 paths $\{P_1, \dots, P_{k_2}\}$ // according to path length
- 18: **Add** nodes and edges of selected paths to DenseGraph(V' , E')
- 19: **end for**
- 20: **end for**
- 21: **Return** DenseGraph(V' , E')

Fig. 5. Pseudo-code for dense graph extraction.

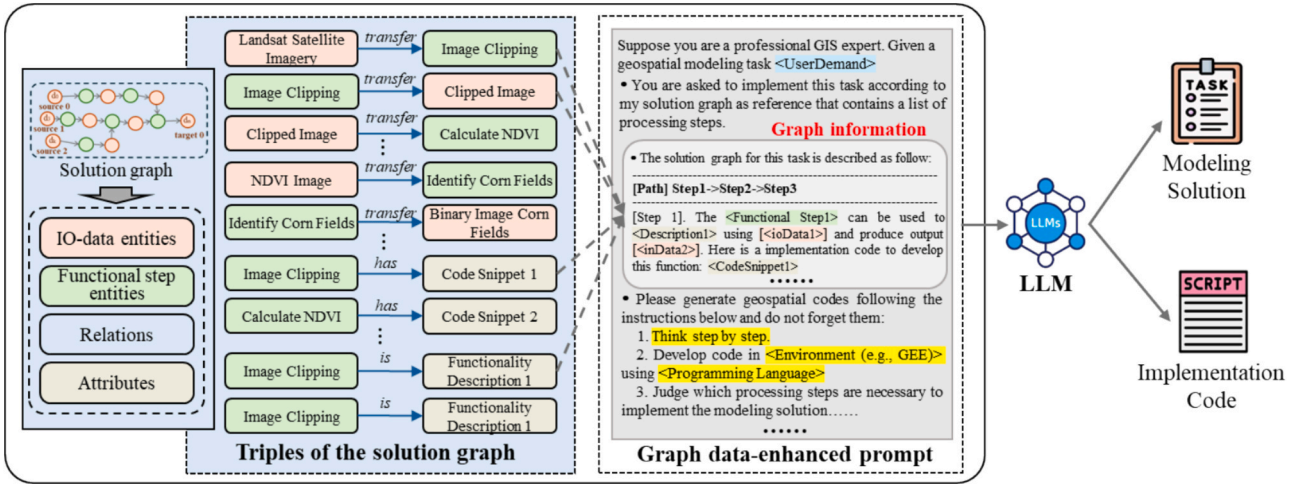


Fig. 6. Process of integrating a solution graph into the prompting template for the solution planning.

connecting source entities to the target entity are encoded as structured phrases and symbols to serve as contextual input. Besides, each functional step entities along the paths, together with its associated IO-data

entities and attributes, is systematically described to provide supplementary guidance for the LLM during generation.

In the prompting template, the LLM is instructed to generate the

modeling solution and corresponding implementation code through the following steps: (1) think step-by-step to construct the modeling solution, which consists of multiple processing steps, with the LLM autonomously determining how to incorporate the provided graph information; (2) describe the relationships between processing steps based on the structural dependencies captured in the solution graph; (3) iteratively assemble the implementation code using a divide-and-conquer approach, with code snippets extracted from the *CodeSnippet* attribute serving as references; (4) verify code syntax by checking variable names, function definitions, logical coherence, and module completeness, and apply necessary corrections.

In addition, the LLM is assigned not only with generating modeling solutions but also with providing explanations that clarify the relevance and consistency of the generated results with respect to the retrieved solution graph, thereby enhancing the robustness and interpretability of the generation. Further implementation details for this module are provided in the Appendix.

4. Experiments

4.1. Experiment setup

4.1.1. Benchmark dataset

Considering the specialized nature of geospatial modeling, we constructed a benchmark dataset by curating two collections of representative samples from prior studies in this domain (Hou et al., 2024a; Liang

et al., 2025). Redundant or functionally similar samples were filtered out, resulting in 100 standardized test cases—each comprising 5 to 10 processing steps—that formed the core of the benchmark. Besides, 200 samples representing simpler tasks with 1 to 4 subsequent steps were also included, bringing the total to 300 samples. Each sample in the benchmark dataset represents a geospatial modeling task executed within the Google Earth Engine environment and consists of six components: raw code, invoked APIs, input data sources, target output, a modeling solution comprising a list of processing steps in natural language, and a query reflecting the task-specific user demand. An overview of these components is provided in Fig. 7. In this study, the benchmark dataset serves as the ground truth for evaluating both the quality of the generated modeling solutions and the correctness of the implementation code.

4.1.2. Competing methods

GeoGraphRAG is examined across LLMs of varying scales and compared against four alternative baseline approaches. First, standard prompting (IO prompt) and CoT prompting are selected to represent methods that rely solely on the intrinsic capabilities of LLMs for geospatial modeling, without the integration of external expert knowledge. These approaches are straightforward and are often preferred by non-expert users. Next, the impact of different graph retrieval strategies on supporting geospatial modeling is also evaluated. The non-parametric retrieval method—the shortest path algorithm, and the LLM-based graph retrieval method, KG-GPT (Sun et al., 2024), are chosen as

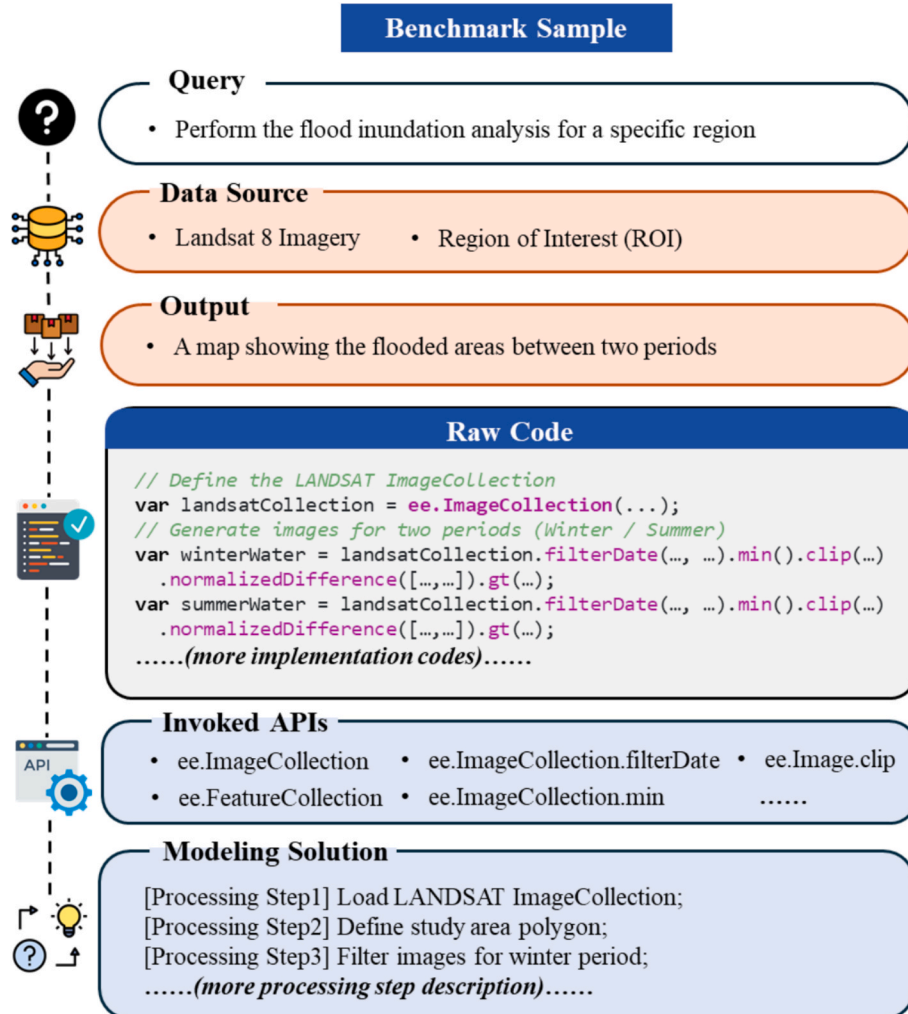


Fig. 7. Example of a sample in the proposed benchmark dataset.

baselines.

Particularly, KG-GPT focuses on making the LLM aware of the neighboring relations of the entity, iteratively selecting relevant relations, and integrating all retrieved triples as contextual prompts to guide LLM reasoning toward generating the response. This method was primarily used for multi-hop knowledge graph question answering (KGQA), where the retrieval process starts from a single entity (the question) and seeks the path leading to the answer. In contrast, GeoGraphRAG places greater emphasis on entity selection and provides the LLM with graph information extracted from a subgraph that is more focused and precise. In addition, in all graph-based retrieval baselines, the *CodeSnippet* attribute associated with each selected functional step—if available—is incorporated into the prompt to assist with code generation. Note that general RAG approaches are not included for comparison, as they rely on task-adaptive retrieval strategies and their knowledge organization and reasoning mechanisms are fundamentally different from those of graph-based RAG approaches. A direct comparison is only appropriate within the context of graph-based RAG methods to ensure fairness.

4.1.3. Evaluation metrics

A set of metrics is applied to quantitatively assess the generated results from GeoGraphRAG and other baselines. Considering the inherent variability and randomness in LLM-generated modeling solutions, traditional methods such as BLEU and ROUGE, or manual assessments are insufficient for effectively and objectively measuring the quality of the results. Therefore, an LLM-based scoring method is adopted to assess the generated modeling solutions along two dimensions: semantic similarity and structural consistency, in terms of their alignment with the standard answer. As LLMs have demonstrated alignment with human judgment, they are employed here as judges to score both dimensions on a scale of 1 to 5, with the final score being the average of all LLM evaluations (Badshah and Sajjad, 2024; J. Gu et al., 2024a). For code implementation, the APIs used in the generated code are compared with the invoked APIs in the ground truth. *Precision* and *Recall* metrics are applied to quantify their alignment, which are computed on a per-sample basis and then averaged across the benchmark. Furthermore, the *F1-score* is applied to balance the trade-off between *Precision* and *Recall*. Note that repeated API calls are not separately counted; instead, we evaluate whether each expected API is present, as the focus is on assessing functional coverage rather than invocation frequency.

4.1.4. S-GMKG & LLMs

In this study, an instance of the S-GMKG is utilized as the external graph-based knowledge source, which integrates geospatial modeling processes extracted from 4820 geoprocessing scripts. Specifically, the schema layer of the S-GMKG is employed, which comprises 15,226 functional step and IO-data entities, as well as 77,138 *invoke* and *transfer* relations. A diverse set of SOTA LLMs of various architectures and scales are applied for GeoGraphRAG and other baseline methods. The selected LLMs include GPT-4o, GPT-3.5, Llama3-70b, Qwen-max, DeepSeek-v3, and DeepSeek-R1-Distill-Qwen-32b (DeepSeek-32b), chosen based on their demonstrated proficiency in geospatial knowledge and their capabilities in code generation.

4.2. Main results

4.2.1. Results for modeling solution generation

The baseline methods are combined with different LLMs and evaluated on samples from the benchmark dataset. The final scores for both semantic similarity and structural consistency are presented in Table 1. The bolded values indicate the highest scores among different methods under the same LLM, while the average score reflects the overall performance of each method across different LLMs. Experimental results show that current SOTA LLMs have already exhibited strong modeling capabilities when using the IO prompt and CoT methods. These LLMs are able to understand user modeling intentions to some extent and generate corresponding modeling solutions without relying on external knowledge. Notably, LLMs such as DeepSeek-32b, enhanced by long CoT mechanism, can even achieve decent performance. Moreover, the results indicate that LLMs overall perform better in geospatial modeling with the assistance of CoT compared to traditional IO prompt approach.

Three graph-based retrieval methods—shortest path, KG-GPT, and GeoGraphRAG—reveal the specialization of geospatial modeling. Although the shortest path method provides a viable route from data sources to the desired output, it may not align with user demands and be even less effective than using IO prompt or CoT methods in several cases. KG-GPT utilizes LLMs to retrieve triples from the graph structure, which is similar to the proposed GeoGraphRAG, but it achieves the lowest performance among all baselines. A potential reason is that KG-GPT processes all possible triples to the LLM, which increases the reasoning cost, and redundant triplets might interfere with their reasoning process. Additionally, KG-GPT is primarily applied in traditional KGQA tasks, where the objective is to obtain a final answer, and the entities accessed through the paths usually serve as auxiliary objects. However, in geospatial modeling tasks, each entity along the path holds equal importance to the target entity, which limits the effectiveness of KG-GPT in current scenarios.

In comparison, the proposed GeoGraphRAG outperforms other baselines in both semantic similarity and structural consistency by narrowing the reasoning scope of LLMs and adopting a stepwise node selection strategy during the graph retrieval. Experimental results show that GeoGraphRAG, when combined with different LLMs, achieves the highest average scores across almost all baselines, demonstrating its broad effectiveness. In particular, GeoGraphRAG with DeepSeek-v3 achieves the best semantic similarity score among all methods. Although GeoGraphRAG does not always secure the top score in structural consistency for every LLM combination, the difference is minimal, which underscores the stability and reliability of its overall performance.

4.2.2. Evaluation for implementation code generation

To further assess the effectiveness of the GeoGraphRAG in supporting geospatial code generation—a core component of geospatial modeling—corresponding implementation code is generated for the benchmark samples. The code is produced by SOTA LLMs based on the modeling solutions presented in Section 4.2.1. To evaluate the correctness of these implementations, the APIs invoked in the generated code are compared against the ground-truth APIs required by each sample.

Table 1

Performance of baselines with different LLMs based on LLM-scored semantic similarity (Sim.) and structural consistency (Con.).

Model	IO Prompt		CoT		Shortest Path		KG-GPT		Ours	
	Sim.	Con.	Sim.	Con.	Sim.	Con.	Sim.	Con.	Sim.	Con.
GPT-4o	3.42	2.70	3.38	2.77	3.40	2.91	3.11	2.48	3.43	2.88
GPT-3.5	3.29	2.52	3.28	2.52	3.14	2.66	3.10	2.41	3.34	2.76
DeepSeek-v3	3.40	2.71	3.48	2.87	3.62	3.17	3.23	2.71	3.65	3.11
qwen-max	3.39	2.67	3.55	2.89	3.59	3.08	3.11	2.51	3.64	3.16
Llama3-70b	3.26	2.50	3.38	2.72	3.33	2.90	2.99	2.45	3.44	2.92
DeepSeek-32b	3.39	2.56	3.36	2.74	3.27	2.76	2.94	2.43	3.40	2.94
Average	3.36	2.61	3.41	2.75	3.39	2.91	3.08	2.50	3.48	2.96

Precision and recall scores for all model-method combinations are summarized in Table 2.

By comparing the performance of different baselines across various LLMs, it is evident that graph retrieval methods offer substantial advantages for geospatial modeling. These methods outperform direct prompting approaches such as IO prompts and CoT, in terms of both *precision* and *recall*. Among the graph retrieval methods, the proposed GeoGraphRAG achieves the highest *recall* across all LLM combinations, indicating its ability to more accurately select and utilize APIs that align with user demands during code generation. Although the *precision* of GeoGraphRAG is slightly lower than that of the other two methods—suggesting that GeoGraphRAG may occasionally introduce unnecessary APIs—we argue that *recall* is a more critical metric, as comprehensive API coverage is vital for ensuring that user demands are fully addressed. To provide a balanced assessment of each method performance, we adopt the *F1-score*, which considers both *precision* and *recall*. As shown in Fig. 8, the proposed method (represented by the blue bar) achieves the highest *F1-score* for 4 out of 6 LLMs and ranks second for the remaining 2, thereby demonstrating overall superior performance compared to all baseline methods.

4.3. Application in landslide susceptibility evaluation

A case study is presented to illustrate the automated geospatial modeling using GeoGraphRAG. Consider a scenario where the decision-maker has access to several geospatial data sources, including region of interest (ROI) data, land use/cover (LULC) imagery, digital elevation model (DEM) data, and multispectral Landsat imagery. His primary goal is to assess landslide susceptibility in a specific region, which is crucial for disaster prevention. The expected output is a detailed landslide susceptibility evaluation map. As shown in Fig. 9, a query expressing the modeling demand is formulated and processed by GeoGraphRAG for demand identification, solution graph retrieval and solution planning, followed by the generation of the corresponding modeling solution and its implementation code using GPT-4o.

During this process, GeoGraphRAG identifies the relevant data sources—such as ROI, LULC imagery, DEM, Landsat imagery, and the landslide susceptibility map—that should be used as the input and output for subgraph retrieval during the demand identification phase. The GPT-4o then interacts with the S-GMKG to map source and target entities and retrieves a solution graph to address user demands, as illustrated in Fig. 10. Based on the solution graph, the processing steps are generated and presented as follows: (1) Landsat imagery is used to calculate the NDVI; (2) DEM imagery is employed to calculate the slope, which is then reclassified into various impact values; (3) a remapping function is applied to the LULC imagery; and (4) the results from the first three steps are weighted and integrated to produce the final landslide susceptibility map. As a result, both the modeling solution and corresponding implementation code for generating the landslide susceptibility map are delivered to the decision-maker.

To further demonstrate the efficiency and necessity of GeoGraphRAG, we employ CoT prompting to generate the implementation code for this landslide susceptibility evaluation model. A comparison of the results in Fig. 11(a) and Fig. 11(b) reveals that by retrieving a

structured solution graph from the S-GMKG, GeoGraphRAG effectively determines how to utilize the different data sources and integrate the intermediate outputs in a predefined manner. The user is only required to make minor adjustments—such as modifying the spatial extent, replacing data sources, or tuning parameter values—within the generated code to produce logically valid and executable scripts. The subsequent steps of code debugging and execution fall outside the scope of the proposed method and are expected to be handled through direct interaction between the user, the LLM, and the code execution environment. In contrast, despite GPT-4o widely regarded as one of the most capable LLMs for reasoning tasks, it fails to produce an effective strategy for combining the intermediate results (e.g., NDVI computation, slope classification) to generate the landslide susceptibility map, even after several rounds of CoT prompting. Comprehensive execution details, including a visual representation of the process, are provided as a demo in the Appendix.

As a result, GeoGraphRAG can effectively address the challenges associated with developing a landslide susceptibility evaluation model by retrieving and leveraging modeling knowledge from the graph. It provides a reliable and interpretable solution for automated geospatial modeling, especially in scenarios where users face knowledge limitations or lack domain expertise.

5. Discussion

The RAG technique has gained widespread recognition for bridging the vertical domain knowledge gaps in LLMs across various fields. By leveraging the structured information between entities, graph data enables more precise and comprehensive retrieval, captures relational knowledge, and generates more accurate, context-aware responses. Inspired by this, the study proposed GeoGraphRAG, which allows LLMs to perform on-demand graph retrieval and utilize graph information for automated geospatial modeling. The experimental results demonstrate the superiority of GeoGraphRAG in both modeling solution and implementation code generation in response to user demands.

Despite the overall improvements, a few instances of output deviation were still observed during the experiments. These deviations were primarily attributed to the model's inherent bias toward selecting paths that appeared plausible but did not align with the intended modeling objectives. Nevertheless, such cases do not constitute factual hallucinations, and their impact is mitigated in this study by employing subgraphs as soft references rather than rigid constraints. Meanwhile, although this study utilizes S-GMKG, a graph structure specifically designed for geospatial modeling, we suggest that the underlying concept and methodology can be extended to other graph-based knowledge sources, thereby enhancing the synergy between graph information and LLMs.

Based on the experiments and our experience, the key contributions of this study for geospatial modeling include:

- (1) Graph information-guided automated geospatial modeling: Conventional geospatial modeling typically relies on manual or semi-automated approaches driven by expert knowledge, or on automatic methods that depend primarily on the geospatial modeling

Table 2

Precision (Prec.) and Recall (Rec.) of comparative approaches in generating implementation code for the benchmark dataset.

Model	IO Prompt Prec.	Rec.	CoT Prec.	Rec.	Shortest Path Prec.	Rec.	KG-GPT Prec.	Rec.	Ours Prec.	Rec.
GPT-4o	0.498	0.703	0.519	0.667	0.664	0.695	0.631	0.674	0.598	0.743
GPT-3.5	0.467	0.635	0.480	0.634	0.640	0.688	0.604	0.621	0.568	0.731
DeepSeek-v3	0.423	0.685	0.451	0.593	0.648	0.689	0.554	0.682	0.650	0.698
qwen-max	0.466	0.565	0.526	0.536	0.604	0.534	0.610	0.561	0.613	0.683
Llama3-70b	0.408	0.489	0.432	0.477	0.562	0.639	0.588	0.611	0.557	0.694
DeepSeek-32b	0.443	0.504	0.449	0.559	0.548	0.554	0.580	0.564	0.576	0.697
Average	0.451	0.597	0.476	0.578	0.611	0.633	0.595	0.619	0.594	0.708

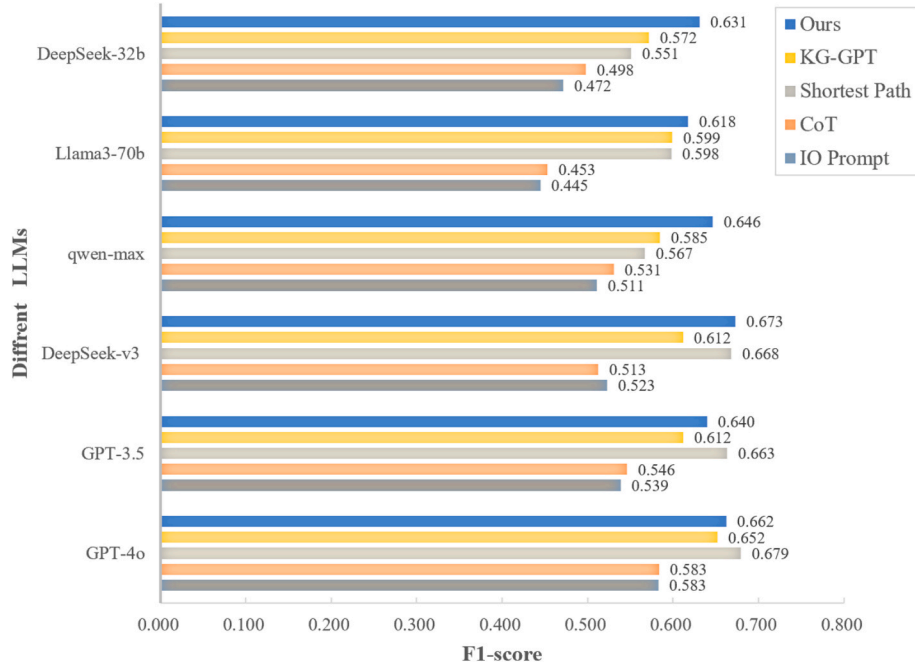


Fig. 8. F1-scores of tested models for geospatial code generation on benchmark samples.

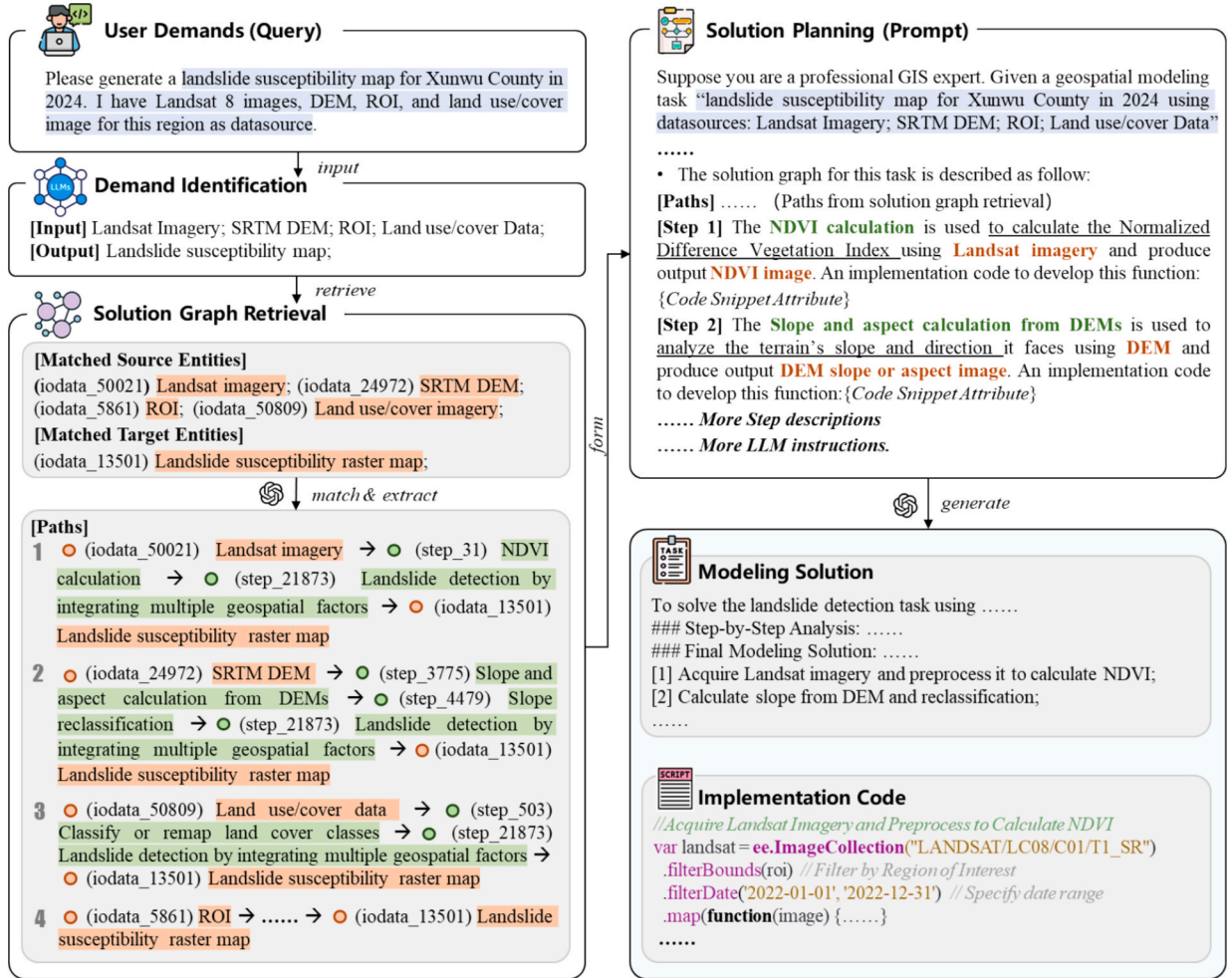


Fig. 9. Intermediate steps and results of modeling landslide susceptibility using GeoGraphRAG.

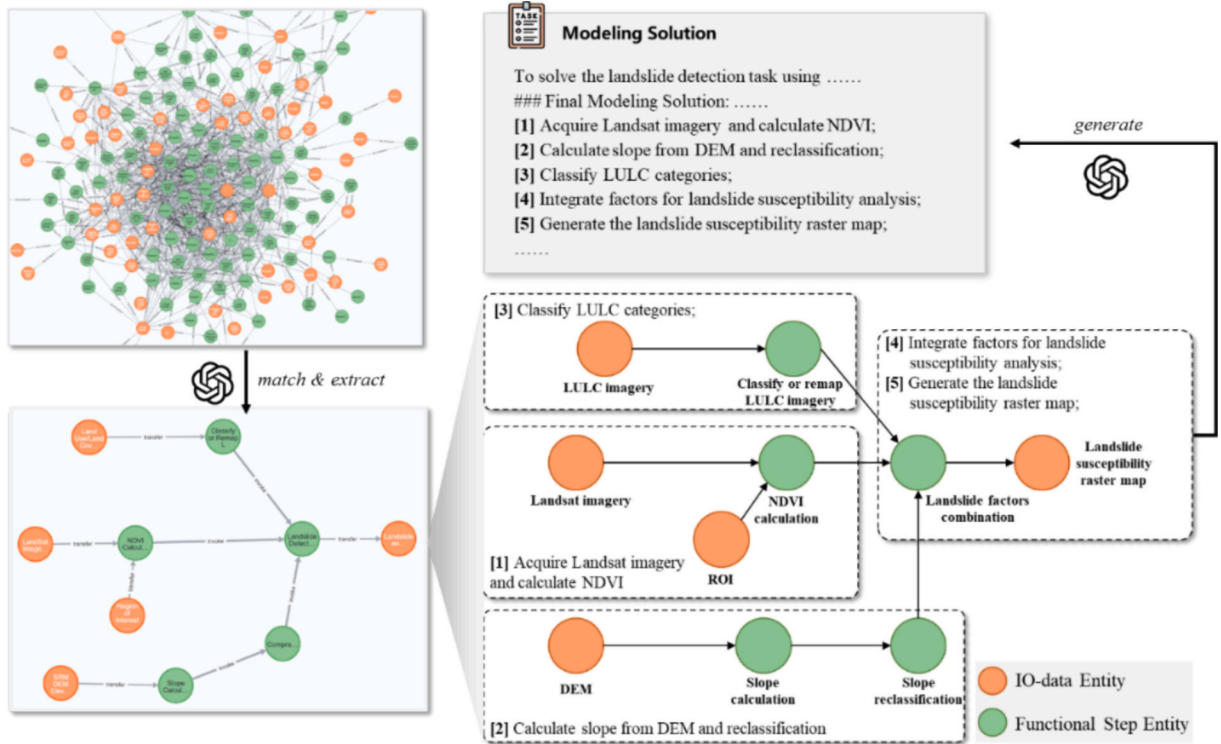


Fig. 10. Visualization of the solution graph extracted from the S-GMKG and explanation of its structure and role in generating the modeling solution.

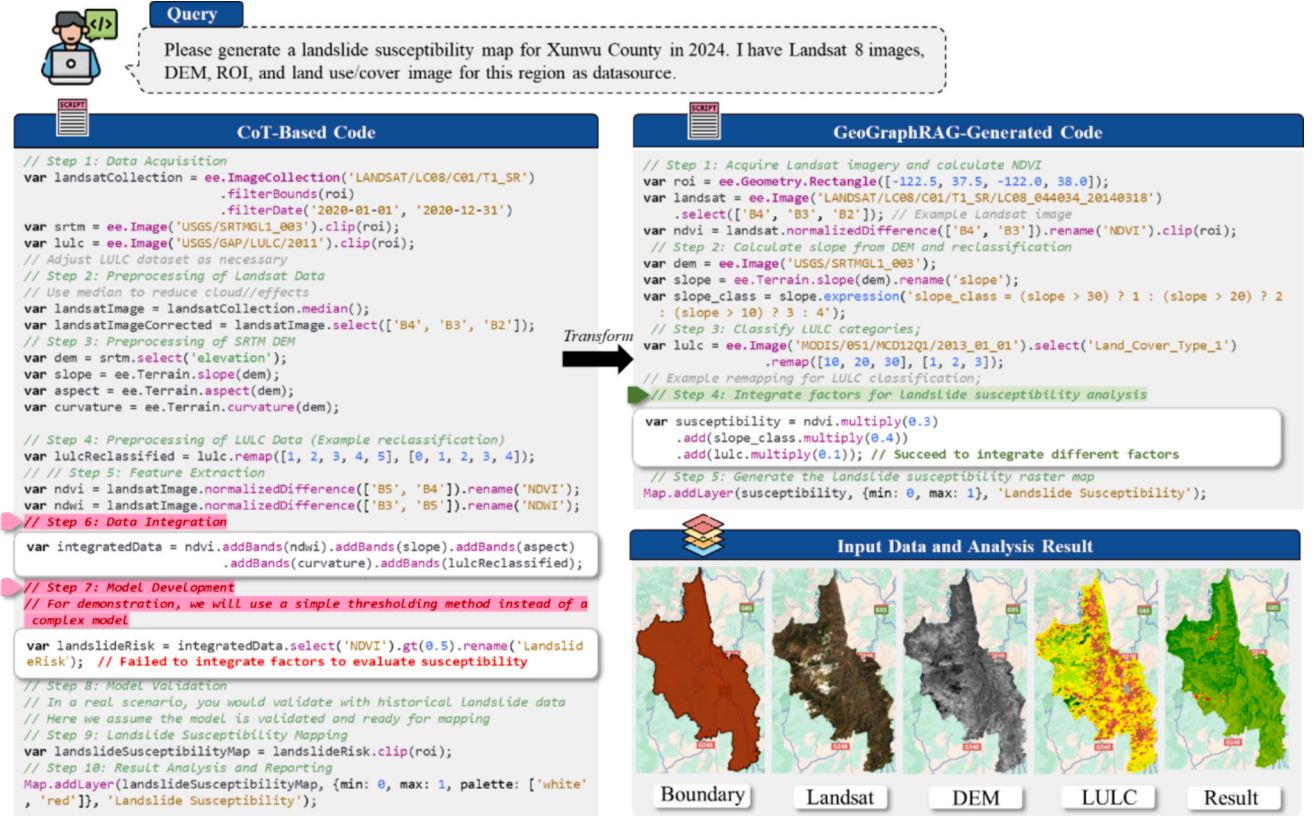


Fig. 11. Comparison of implementation codes generated by CoT prompting and GeoGraphRAG.

capabilities of LLMs. In contrast, GeoGraphRAG retrieves the necessary knowledge from the prior-existing graph knowledge base (S-GMKG), guiding LLMs to generate customized modeling

solutions and implementation code for specific tasks. This approach strikes a balance between the generative abilities of LLMs and the domain expertise traditionally required for

geospatial modeling, mitigating hallucination issues caused by insufficient domain knowledge in LLMs and their lack of awareness of their own knowledge boundaries. Meanwhile, reliance on expert experience is also minimized to articulating the modeling intent. Once a clear description with the input and output specifications is provided, the corresponding processing steps can be automatically retrieved from the S-GMKG without manual intervention, and the generation of modeling solutions and implementation code is accordingly fully automated.

- (2) Interpretable geo-analysis model generation: GeoGraphRAG can partially guarantee the explainability of the geo-analysis model generation procedure by grounding the modeling solution in a subgraph retrieved from the S-GMKG. Since the modeling solution is derived from this subgraph, where each functional step and its associated relationships have been previously constructed and validated by domain experts, both the process and the rationale behind the code generated by LLMs can be explained. Moreover, when the reference code for a functional step, stored in the *CodeSnippet* attribute, is integrated into the prompt, it helps guide the LLM toward correct API usage and a more reasonable step sequence, thereby minimizing the risk of calling inappropriate functions or producing disordered code blocks. Even if errors occur in the final code implementation, they can often be corrected by comparing the output with the retrieved subgraph, making the overall process both interpretable and easier to debug.
- (3) Low-barrier collaborative reuse of geospatial modeling knowledge: GeoGraphRAG enables the flexible retrieval of subgraphs from the S-GMKG, facilitating the reuse of individual modeling processes or combinations of multiple processes within the graph structure. By leveraging the synergy between the graph information and the LLM, geospatial modeling becomes a process of reusing composable “knowledge blocks”, which improves both its modularity and scalability, and provides an effective mechanism for adapting existing models to new contexts. Additionally, this approach accelerates the dissemination of existing modeling experience and can foster efficient collaboration among researchers from diverse backgrounds. These capabilities are particularly valuable in application scenarios such as urban planning, disaster management and decision making (Fang et al., 2023; Li and Ning, 2023). As a result, GeoGraphRAG contributes to a more dynamic and inclusive geospatial modeling ecosystem, where knowledge is continuously exchanged and expanded.

Admittedly, this study still has several limitations. Although an ambiguity-based user demand categorization approach was introduced for user intent understanding, challenges remain in handling implicit expressions, vague statements, and non-textual demand types. Future work could incorporate a more comprehensive domain knowledge system and support multimodal inputs (e.g., map interactions, image captions) to broaden applicability across diverse query scenarios. The proposed approach relies on the inherent capabilities of LLMs to select entities from S-GMKG for constructing solution graphs. Recent studies, including GNN-based subgraph retrieval methods (Mavromatis and Karypis, 2024) and hybrid models that combine GNN reasoning with LLM-driven semantic understanding (M. Zhang et al., 2024), offer alternative approaches. Moreover, the current evaluation focuses on semantic similarity and API usage without verifying the actual execution outcomes of the generated code. Due to the lack of standardized output references in the benchmark dataset, execution-based evaluation was not feasible, and an alignment-based strategy was adopted. Future research may consider the use of curated tasks with well-defined outputs to enable more comprehensive validation. In addition, the spatial knowledge embedded in the generated code has the potential to enrich the underlying knowledge graph. Future efforts will explore interactive update mechanisms and strengthen the integration of dynamic

knowledge graph updates with spatial databases to foster deeper collaboration between language models and structured geospatial knowledge systems.

6. Conclusion

This study presents GeoGraphRAG, a graph-based RAG approach that retrieves graph information on demand from the external graph knowledge base—S-GMKG—to enable automated geospatial modeling. GeoGraphRAG is evaluated against comparative baselines using a series of LLMs on the geospatial modeling benchmark dataset. A typical application case is presented to illustrate how GeoGraphRAG automates the generation of modeling solution and implementation code in response to specific user demands in a clear and interpretable manner.

Based on the experimental results, this study suggests that the proposed approach provides a promising direction for advancing the automation, interpretability, and accessibility of geospatial modeling. GeoGraphRAG integrates graph-structured knowledge with LLM-based reasoning to support dynamic and flexible automated modeling; enables the generation of interpretable geo-analysis models that enhance user understanding, transparency, and trust in the modeling outcomes; and facilitates low-barrier, collaborative reuse and continuous adaptation of geospatial modeling knowledge, which promotes broader accessibility and knowledge sharing across application domains.

The future work will build upon GeoGraphRAG and focus on the following key directions: deeper exploration of user demands, development of data-driven graph-based RAG approaches, incorporation of execution-based evaluation strategies and advancement of dynamic knowledge graph updating and spatial database integration to foster tighter bidirectional synergy between LLMs and graph structures. We believe that these efforts will further promote the application of LLMs in geospatial modeling and contribute to delivering greater value for the broader geospatial community.

Funding

This research was supported by the National Key Research and Development Program of China (Grant No. 2024YFB3909603) and the National Natural Science Foundation of China (Grant No. 41930107).

CRediT authorship contribution statement

Jianyuan Liang: Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Shuyang Hou:** Writing – original draft, Resources, Data curation. **Haoyue Jiao:** Visualization, Validation. **Yaxian Qing:** Visualization, Validation, Resources. **Anqi Zhao:** Investigation, Formal analysis. **Zhangxiao Shen:** Validation. **Longgang Xiang:** Funding acquisition. **Huayi Wu:** Writing – review & editing, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We would like to express our sincere gratitude to the reviewers and editors for their valuable comments that help improve this paper. This research was supported by the National Key Research and Development Program of China (Grant No. 2024YFB3909603) and the National Natural Science Foundation of China (Grant No. 41930107).

Data availability

Data will be made available on request.

References

- Badshah, S., Sajjad, H., 2024. Reference-Guided Verdict: LLMs-as-Judges in Automatic Evaluation of Free-Form Text. *arXiv Prepr. arXiv2408.09235*.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., 2020. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* 33, 1877–1901.
- Cao, Y., Li, S., Liu, Y., Yan, Z., Dai, Y., Yu, P.S., Sun, L., 2023. A comprehensive survey of ai-generated content (aigc): A history of generative ai from gan to chatgpt. *arXiv Prepr. arXiv2303.04226*.
- Chein, M., Mugnier, M.-L., 2008. Graph-based knowledge representation: computational foundations of conceptual graphs. Springer Science & Business Media.
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z., 2024a. Bge m3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings through Self-Knowledge Distillation. *arXiv Prepr. arXiv2402.03216*.
- Chen, M., Qian, Z., Boers, N., Creutzig, F., Camps-Valls, G., Hubacek, K., Claramunt, C., Wilson, J.P., Nativi, S., Jakeman, A.J., 2024b. Collaboration between artificial intelligence and Earth science communities for mutual benefit. *Nat. Geosci.* 17, 949–952.
- Chen, M., Voinov, A., Ames, D.P., Kettner, A.J., Goodall, J.L., Jakeman, A.J., Barton, M. C., Harpham, Q., Cuddy, S.M., DeLuca, C., Yue, S., Wang, J., Zhang, F., Wen, Y., Lü, G., 2020. Position paper: Open web-distributed integrated geographic modelling and simulation to enable broader participation and applications. *Earth-Sci. Rev.* <https://doi.org/10.1016/j.earscirev.2020.103223>.
- Delile, J., Mukherjee, S., Van Pamel, A., Zhukov, L., 2024. Graph-Based Retriever Captures the Long Tail of Biomedical Knowledge, in: ICML'24 Workshop ML for Life and Material Science: From Theory to Industry Applications.
- Deng, C., Zhang, T., He, Z., Chen, Q., Shi, Y., Xu, Y., Fu, L., Zhang, W., Wang, X., Zhou, C., 2024. K2: A foundation language model for geoscience knowledge understanding and utilization. In: *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pp. 161–170.
- Fang, Z., Yue, P., Zhang, M., Xie, J., Wu, D., Jiang, L., 2023. A service-oriented collaborative approach to disaster decision support by integrating geospatial resources and task chain. *Int. J. Appl. Earth Obs. Geoinf.* 117, 103217.
- Gao, F., Yue, P., Cao, Z., Zhao, S., Shanguan, B., Jiang, L., Hu, L., Fang, Z., Liang, Z., 2022. A multi-source spatio-temporal data cube for large-scale geospatial analysis. *Int. J. Geogr. Inf. Sci.* 36, 1853–1884.
- Gramacki, P., Martins, B., Szymański, P., 2024. Evaluation of Code LLMs on Geospatial Code Generation, in: *Proceedings of the 7th ACM SIGSPATIAL International Workshop on AI for Geographic Knowledge Discovery*, pp. 54–62.
- Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., 2024a. A Survey on Llm-as-a-Judge. *Arxiv Prepr. arXiv2411.15594*.
- Gu, X., Chen, M., Lin, Y., Hu, Y., Zhang, H., Wan, C., Wei, Z., Xu, Y., Wang, J., 2024b. On the effectiveness of large language models in domain-specific code generation. *ACM Trans. Softw. Eng. Methodol.*
- He, Y., Chen, M., Wen, Y., Yue, S., Lü, G., 2023. A web-based strategy to reuse grids in geographic modeling. *Int. J. Appl. Earth Obs. Geoinf.* 116, 103170.
- Hou, S., Jiao, H., Shen, Z., Liang, J., Zhao, A., Zhang, X., Wang, J., Wu, H., 2024a. Chain-of-programming (CoP)empowering large language models for geospatial code generation. *Arxiv Prepr. arXiv2411.10753*.
- Hou, S., Shen, Z., Zhao, A., Liang, J., Gui, Z., Guan, X., Li, R., Wu, H., 2025. GeoCode-GPT: A large language model for geospatial code generation. *Int. J. Appl. Earth Obs. Geoinf.* 104456. <https://doi.org/10.1016/j.jag.2025.104456>.
- Hou, S., Zhangxiao, S., Jianyuan, L., Anqi, Z., Zhipeng, G., Rui, L., Wu, H., 2024b. Can Large Language Models Generate Geospatial Code? *arXiv Prepr. arXiv2410.09738*.
- Jin, B., Xie, C., Zhang, J., Roy, K.K., Zhang, Y., Wang, S., Meng, Y., Han, J., 2024. Graph chain-of-thought: Augmenting large language models by reasoning on graphs. *CoRR*.
- Kim, J., Kwon, Y., Jo, Y., Choi, E., 2023. KG-GPT: A general framework for reasoning on knowledge graphs using large language models. *arXiv Prepr. arXiv2310.11220*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv. Neural Inf. Process. Syst.* 33, 9459–9474.
- Li, D., Yang, S., Tan, Z., Baik, J.Y., Yun, S., Lee, J., Chacko, A., Hou, B., Duong-Tran, D., Ding, Y., 2024. DALK: Dynamic Co-Augmentation of LLMs and KG to Answer Alzheimer's Disease Questions with Scientific Literature. *Arxiv Prepr. arXiv2405.04819*.
- Li, H., Zhang, C., Xiao, Z., Chen, M., Lu, D., Liu, S., 2021. A Web-based geo-simulation approach integrating knowledge graph and model-services. *Environ. Model Softw.* 144, 105160. <https://doi.org/10.1016/j.envsoft.2021.105160>.
- Li, Z., Ning, H., 2023. Autonomous GIS: The next-generation AI-powered GIS. *Int. J. Digit. Earth* 16, 4668–4686.
- Liang, J., Hou, S., Zhao, A., Xu, Q., Xiang, L., Li, R., Wu, H., 2025. Design and application of a semantic-driven geospatial modeling knowledge graph based on large language models. *Geo-Spatial Inf. Sci.*
- Liang, J., Zhao, A., Hou, S., Jin, F., Wu, H., 2024. A GPT-enhanced framework on knowledge extraction and reuse for geographic analysis models in Google Earth Engine. *Int. J. Digit. Earth* 17, 2398063. <https://doi.org/10.1080/17538947.2024.2398063>.
- Lin, Z., Deng, C., Zhou, L., Zhang, T., Xu, Y.I., Xu, Y., He, Z., Shi, Y., Dai, B., Song, Y., 2023. Geogalactica: A scientific large language model in geoscience. *arXiv Prepr. arXiv2401.00434*, ..
- Liu, F., Liu, Y., Shi, L., Huang, H., Wang, R., Yang, Z., Zhang, L., 2024. Exploring and Evaluating Hallucinations in Llm-Powered Code Generation. *Arxiv Prepr. arXiv2404.00971*.
- Luo, L., Li, Y.-F., Haf, R., Pan, S., 2024. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning, in: *The Twelfth International Conference on Learning Representations*.
- Manvi, R., Khanna, S., Mai, G., Burke, M., Lobell, D., Ermon, S., 2023. Geollm: Extracting geospatial knowledge from large language models. *arXiv Prepr., arXiv2310.06213*.
- Mavromatis, C., Karypis, G., 2024. Gnn-rag: Graph neural retrieval for large language model reasoning. *arXiv Prepr. arXiv2405.20139*.
- Mooney, P., Cui, W., Guan, B., Juhász, L., 2023. Towards understanding the geospatial skills of chatgpt: Taking a geographic information systems (gis) exam, in: *Proceedings of the 6th ACM SIGSPATIAL International Workshop on AI for Geographic Knowledge Discovery*, pp. 85–94.
- Ning, H., Li, Z., Akinboyewa, T., Lessani, M.N., 2025. An autonomous GIS agent framework for geospatial data retrieval. *Int. J. Digit. Earth* 18, 2458688.
- Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., Wu, X., 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Trans. Knowl. Data Eng.*
- Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., Tang, S., 2024. Graph Retrieval-Augmented Generation: A Survey. *Arxiv Prepr. arXiv2408.08921*.
- Robinson, I., Webber, J., Eifrem, E., 2015. Graph databases: new opportunities for connected data. “O'Reilly Media, Inc”.
- Sun, J., Xu, C., Tang, L., Wang, S., Lin, C., Gong, Y., Ni, L., Shum, H.-Y., Guo, J., 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph, in: *The Twelfth International Conference on Learning Representations*.
- Wei, C., Zhang, Y., Zhao, X., Zeng, Z., Wang, Z., Lin, J., Guan, Q., Yu, W., 2024. GeoTool-GPT: A trainable method for facilitating Large Language Models to master GIS tools. *Int. J. Geogr. Inf. Sci.* 1–25.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., 2022. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* 35, 24824–24837.
- Wu, H., Zhao, A., Liang, J., Hou, S., 2024. Five-layer hierarchical network (5-HiNet) of geospatial information service for AIGC of geographic analysis model. *Acta Geod. Cartogr. Sin.* 53, 2053.
- Xia, Y., Huang, Y., Qiu, Q., Zhang, X., Miao, L., Chen, Y., 2024. A question and answering service of typhoon disasters based on the t5 large language model. *ISPRS Int. J. Geo-Information* 13, 165.
- Zhang, M., Sun, M., Wang, P., Fan, S., Mo, Y., Xu, X., Liu, H., Yang, C., Shi, C., 2024. Graphtranslator: Aligning graph model to large language model for open-ended tasks, in: *Proceedings of the ACM Web Conference 2024*, pp. 1003–1014.
- Zhang, Y., Wang, Z., He, Z., Li, J., Mai, G., Lin, J., Wei, C., Yu, W., 2024b. BB-GeoGPT: A framework for learning a large language model for geographic information science. *Inf. Process. Manag.* 61, 103808.
- Zhang, Y., Wei, C., He, Z., Yu, W., 2024c. GeoGPT: An assistant for understanding and processing geospatial tasks. *Int. J. Appl. Earth Obs. Geoinf.* 131, 103976.